

From LangChain to langgraphR

Simple speaker notes

What LangChain is, why LangGraph was made, and how langgraphR brings it to R.

A companion to the slide deck. Each part explains one slide in plain, casual words you can say out loud.

SLIDE 1 · COVER

From LangChain to langgraphR

Welcome. This is a simple story in three parts.

Part one: what LangChain is — in plain English, no jargon.

Part two: the problem. LangChain only goes in a straight line. Real AI needs to loop, branch, and wait. That's why LangGraph was made.

Part three: how we bring that to R. LangGraph is Python-only, and R can't run it. So we built langgraphR — a thin R wrapper over a hidden Python server.

By the end, you'll know what each piece does, and whether it's a good idea to build AI in R this way.

SLIDE 2 · SECTION ONE

The Foundation

Let's start from the beginning, with LangChain.

The idea to keep in mind: LangChain made it easy to build AI apps — but only as long as they moved in a straight line. I'll explain both halves of that.

SLIDE 3

What is LangChain and how it works

By themselves, AI models are simple. You give them text, they give you text back. That's it. No memory, no files, no tools. All of that has to be added by you.

LangChain wraps stuff around the model. It connects it to your documents, your database, past messages, and tools. And it chains steps together into a workflow.

But honestly? Under the hood it just does four small things, over and over. One: turn your input into a prompt. Two: call the model's API. Three: get the reply back and clean it up. Four: pass it to the next step.

That's literally it. It's a wrapper around API calls, plus some glue.

Example. A user says: 'My order hasn't arrived in 2 weeks.' LangChain wraps that into a full prompt, like 'You are a helpful support agent...'. It calls the API. The model replies. LangChain cleans up the reply. Then it feeds that reply to the next step — maybe 'now translate this into French.'

Small steps, chained together. That's LangChain. Remember this, because the problem with chains is coming next.

SLIDE 4

Where chains hit a wall

There was a real problem.

LangChain goes in a straight line — step 1, step 2, step 3. That's fine for simple stuff. But real agents need more.

First: what if it needs to loop back and retry a step? A chain can't go backward.

Second: what if it needs to branch — do this if X, do that if Y? A chain only knows one next step.

Third: what if it needs to pause and wait for a human to say OK? Imagine it's about to send a refund. You want it to stop. A chain can't stop — it runs to the end.

LangChain couldn't handle any of this, because it only moves forward. So they made LangGraph, which models your workflow as a graph — nodes and edges. Now you can loop, branch, and decide what comes next.

In short: LangChain was too rigid. LangGraph fixed it.

SLIDE 5 · SECTION TWO

LangGraph

So the fix was simple: stop thinking in lines, start thinking in graphs. That's LangGraph.

SLIDE 6

LangGraph — nodes, edges, and loops

LangGraph builds your AI as a graph. That means it can loop, branch, and make decisions.

Three simple ideas. Nodes are steps — call the AI, run a tool. Edges say what comes next, and when. And the AI moves around, able to go forward, loop back, or switch paths.

Example: a support bot. A user complains. The AI reads it — is this billing or technical? If billing, draft a billing reply. If technical, draft a technical reply. If unclear, loop back and ask the user a question. Then send the final answer.

That 'loop back' step is the whole point. A chain can't do it — it would push forward even when unsure. LangGraph can loop back. That's what makes it feel like a real agent.

SLIDE 7

LangGraph in Python vs. R

Here's where the talk turns. LangGraph has no R version. It's Python and JavaScript only.

So can you rebuild it in R? The answer splits in two.

The graph part is easy in R. Nodes, edges, state, loops, memory — all just data and functions. R does that fine.

But the engine part is hard. Parallel work, threads, streaming, pausing, fan-out — all difficult or impossible in plain R. The table shows exactly where it flips from 'yes' to 'hard'.

So: can you describe a graph in R? Yes. Can you build the full runtime in R? Very hard, almost not worth it. That gap is the whole reason langgraphR exists — it bridges it with a Python server.

LangGraph requirement	Python	Pure R
Graph / nodes / edges	Yes	Yes
State management	Yes	Yes
Reducers	Yes	Yes
Channels	Yes	Yes
Conditional routing	Yes	Yes
Loops / cycles	Yes	Yes
Subgraphs	Yes	Yes
Pregel / BSP execution	Yes	Difficult
Parallel execution	Yes	Difficult
Thread-level concurrency	Yes	No
Async execution	Yes	Difficult
Task scheduling	Yes	Difficult
Fine-grained cancellation	Yes	No
Dynamic task spawning	Yes	Difficult
Fan-out / fan-in	Yes	Difficult
Checkpointing	Yes	Yes
Persistence	Yes	Yes
Interrupt / resume	Yes	Difficult
Streaming	Yes	Difficult
Runtime context	Yes	Yes
Memory / store	Yes	Yes
Caching	Yes	Yes
Serialization	Yes	Yes
Tool execution	Yes	Yes
Message handling	Yes	Yes
Debugging / inspection	Yes	Yes
Durable execution	Yes	Difficult
Distributed execution	Yes	Difficult
High-performance execution engine	Yes	Difficult
Can you implement the basic graph concepts?	Yes	Yes
Can you reproduce the complete runtime purely in the language?	Yes	Very difficult

SLIDE 8

Why you can't build this purely in R

So why not just rewrite it in R? Because the hard parts are things R was never built to do. Five reasons:

One: it runs on Python's async engine — an event loop that juggles many tasks at once. R has nothing like it. You'd have to build the engine first.

Two: no streaming. A fast agent shows words one by one. Base R has no HTTP client that can read that stream, so you can't show the first word until the whole answer is done.

Three: it needs to run 24/7. R is made for short, one-at-a-time sessions — the opposite of what an orchestrator needs.

Four: you'd be rewriting a whole framework, then keeping it in sync with Python forever.

Five: the ecosystem is Python. Models, tools, tracing, monitoring — all Python-first.

Bottom line: you wouldn't be writing an R app. You'd be maintaining a Python port forever. Not worth it.

SLIDE 9

What about Rcpp — a C++ engine?

People always ask: what about Rcpp? You can write C++ and call it from R.

And to be fair, C++ fixes a lot. Threads, async, scheduling, queues, thousands of tasks — C++ does all that natively. The table shows it: the bolded bottom rows are where C++ wins.

But two rows stay 'partial' even with C++: canceling running R code, and killing R from another thread. R itself isn't thread-safe, so C++ can't fully fix that.

And here's the real point. Even if you finished, you'd still need to hook up the model APIs and build streaming yourself — the exact stuff R lacks. You'd do the hardest work to end up where LangGraph already is, for free.

Rewriting a proven Python runtime in C++ buys you nothing. Better to keep the Python and wrap it in R. That's what langgraphR does.

Capability	Pure R	C++ + Rcpp	Why it matters
Graph / nodes / edges	Yes	Yes	Normal data structures
State management	Yes	Yes	R handles this well
Reducers	Yes	Yes	Functions + data
Channels	Yes	Yes	Can be implemented
Conditional routing	Yes	Yes	Normal control flow
Loops / cycles	Yes	Yes	Normal control flow
Subgraphs	Yes	Yes	Normal structures
Checkpointing	Yes	Yes	R can write/read state
Persistence	Yes	Yes	Databases/files work
Serialization	Yes	Yes	R has serialization
Memory / store	Yes	Yes	Can use DB/KV systems
Caching	Yes	Yes	Fully implementable
Tool execution	Yes	Yes	R can call tools/processes
Message handling	Yes	Yes	Fully implementable
Runtime context	Yes	Yes	Fully implementable
True high-performance thread pool	Partial	Yes	C++ can own native threads
Fine-grained thread-level concurrency	No	Yes	Major R runtime limitation
Native async runtime	Partial	Yes	C++ can manage event/task execution
Reliable cancellation of arbitrary running R code	No	Partial	Important limitation remains
Kill/interruption of arbitrary R execution from another thread	No	Partial	R itself is not generally thread-safe
Shared mutable state across R threads	No	Yes*	C++ can safely manage its own state
Lock-free / atomic C++ data structures	No	Yes	Native C++ capability
Condition variables / native synchronization	No	Yes	C++ runtime primitives
Low-overhead concurrent queues	No	Yes	Native C++
Thousands of lightweight native tasks	No	Yes	C++ runtime can manage them
Pregel-style native scheduler	Partial	Yes	C++ is much better suited
Precise task scheduling/control	Partial	Yes	C++ controls workers directly
Background C++ workers independent of R evaluation	No	Yes	Major architectural advantage
Native parallel execution of the C++ engine itself	No	Yes	C++ can run independently
High-performance concurrent graph runtime	No	Yes	This is the big one

SLIDE 10

Bridges and wrappers — reaching other languages from R

R doesn't have to stay in R. Two little ideas unlock everything: bridges and wrappers.

A bridge connects R to another language and translates between them. You write R, it's forwarded to Python (or Java, C++), and the results come back as R objects. The two languages talk to each other.

A wrapper is a layer of R functions on top of a foreign library. You call the R function, and it quietly forwards to the real library underneath. You never write the other language yourself.

The example is reticulate. It's an R package that runs Python inside R. So you write R, but you can call Python libraries — like LangGraph — directly from your R script.

SLIDE 11

langgraphR — what it is and how it works

langgraphR is an R package that wraps LangGraph. You describe the agent as a graph in plain R, and a hidden Python server runs the real LangGraph underneath. You get loops, memory, and human approval without leaving R.

Four layers, top to bottom.

Your R code: load library(langgraphr), then describe the agent with lg_graph(), lg_add_node(), lg_add_edge() — or just chat through an object called LgAgent.

The R client: lg_connect() starts the hidden Python server (FastAPI) if it isn't already running. You never manage Python yourself.

The Python sidecar: it turns your R graph into a real LangGraph graph. When a node needs R code, it interrupts, R runs your function, and the run resumes. Your R functions never cross the wire as code.

Model + memory: DeepSeek or OpenAI answer the LLM calls, and SQLite checkpoints save state so runs survive a crash.

That's the whole machine.

SLIDE 12

How one graph\$invoke() call works, step by step

Last slide — a single run, step by step. This is the heartbeat.

Step 1: you write the graph in R. Each step is one of your R functions — that's a node.

Step 2: lg_compile() packages it into a blueprint and sends it to Python.

Step 3: graph\$invoke('question') starts the run.

Step 4: Python starts, but it can't run R. So it pauses at the first node and sends the state back to R.

Step 5: R runs your node function. It can call the LLM with `lg_call_model()`.

Step 6: R sends back the updated state, plus which node is next.

Step 7: Python checks — another node, or done? If another, it loops back to step 5.

Step 8: when a node says it's done, Python sends the final state back.

Step 9: you read the answer — `res$state$answer`.

Simple rhythm: Python runs the graph, and every time it hits an R function it pauses and hands control over. Your logic stays in R.

That's the whole story. LangChain made straight lines easy. LangGraph replaced the line with a graph. And langgraphR brings that graph to R through a thin wrapper over a hidden Python server.